

Solution Architecture

Date	18 September 2026
Team ID	SWTID-2026-2825
Project Name	AI WeatherWise
Maximum Marks	5 Marks

Solution Architecture Diagram

AI WeatherWise follows a layered REST architecture: a client/API tool sends requests through the Express gateway, which routes them (after JWT verification on private routes) to Controllers, which delegate business logic to Services, which call either MongoDB or the external OpenWeatherMap / Google Gemini APIs.

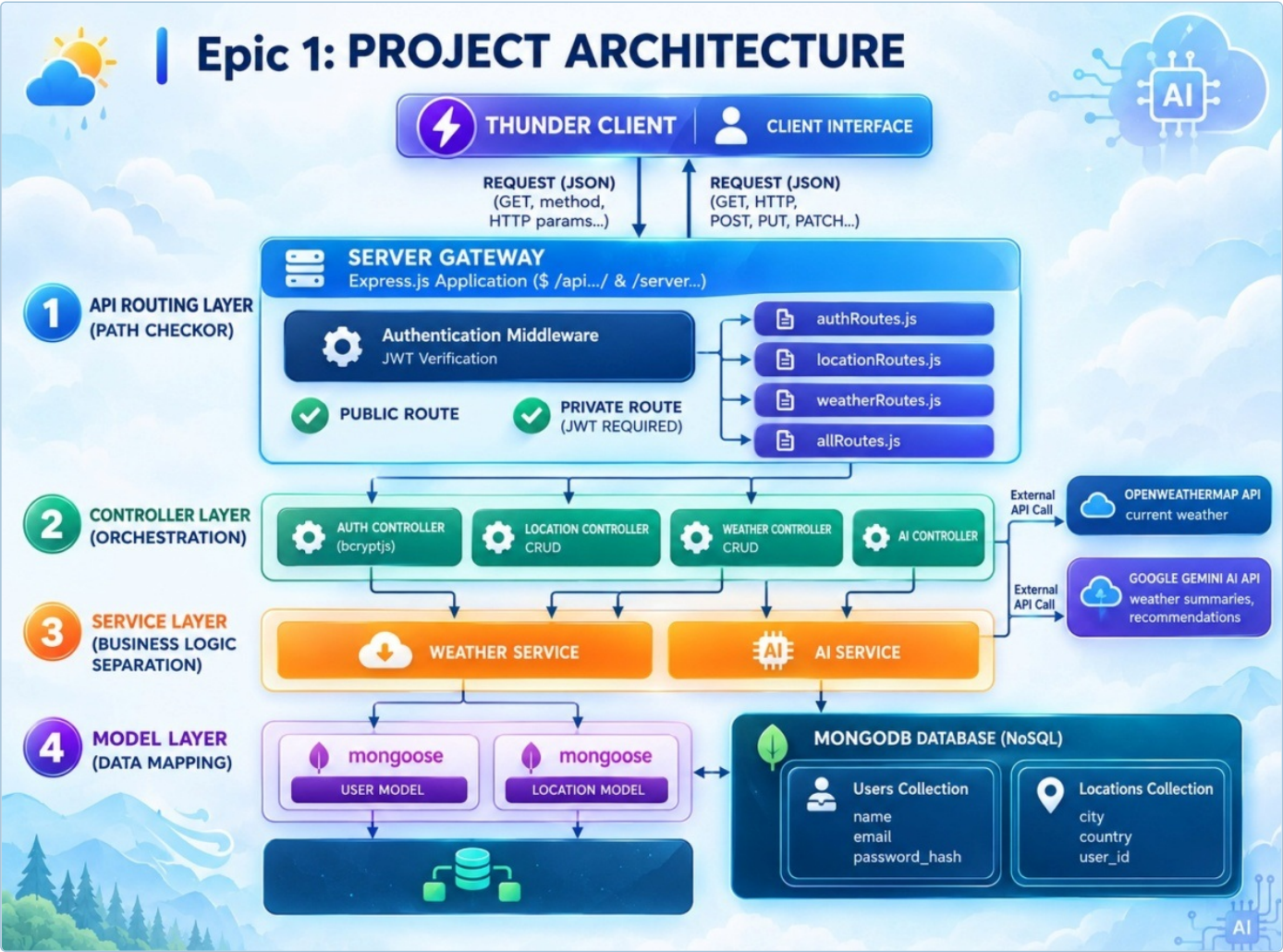
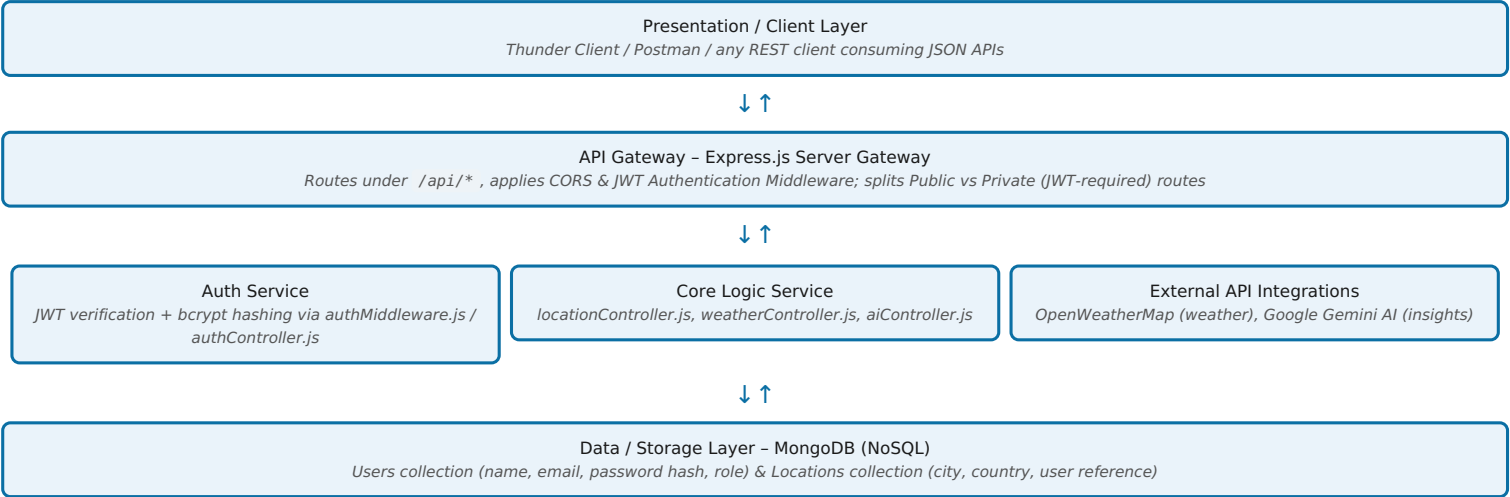


Fig. 1 – Epic 1 project architecture as implemented (Thunder Client → Express Gateway → Controllers → Services → MongoDB / external APIs).

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	REST client used to send and inspect JSON requests/responses during development and grading	Thunder Client, Postman
API Gateway	Central Express.js entry point; parses JSON, applies CORS, dispatches to auth/location/weather/ai route groups	Express.js, cors, authMiddleware (JWT)

Microservices (Controllers / Services)	Encapsulate domain logic: authController (register/login), locationController (favorite CRUD), weatherController + weatherService (OpenWeatherMap), aiController + aiService (Gemini insights)	Node.js, Express Router, @google/genai
Database	Persists Users and Locations with Mongoose-enforced schema validation and a one-to-many relation	MongoDB, Mongoose

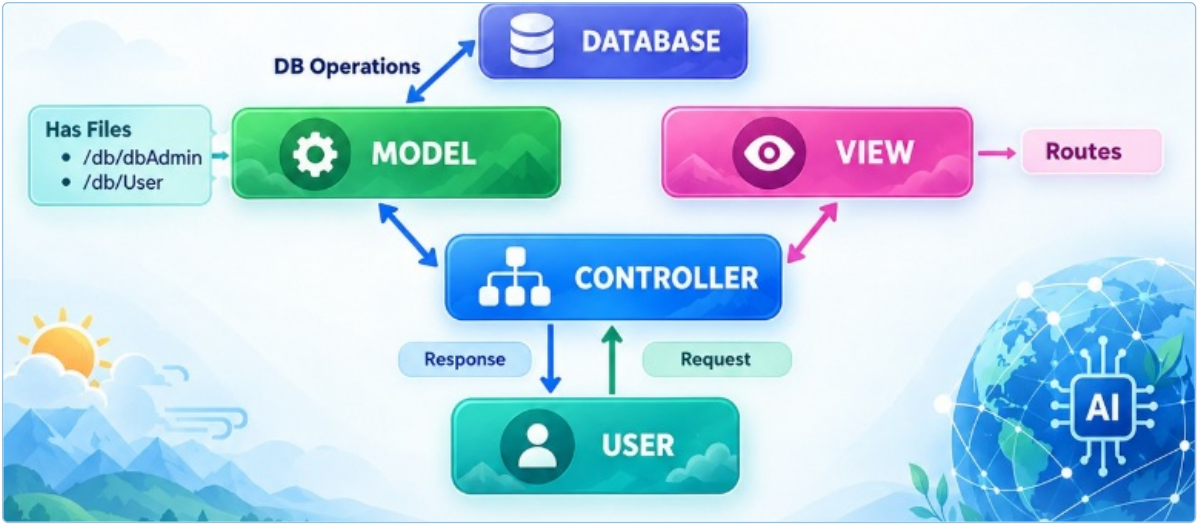


Fig. 2 – MVC pattern used within the Core Logic Service: Model (Mongoose schemas) ↔ Controller (route handlers) ↔ View (JSON routing layer, since this is a headless API).